Prioritized Booking of Autonomous Cars in Organization with Kona's Algorithm

Dr D. Haritha[1*] B.N. Nithin Chakravarthi[1] G. Lalith Kumar Reddy, K.Sravya[1]

[1*] Professor, [1] Student

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Vaddeswaram, AP, India.

pedduharitha@gmail.com, lalithkumarreddy03@gmail.com, lohithakona29@gmail.com, bswapnakalyan@gmail.com,

Abstract –

Detecting the shortest and optimal path plays a vital role when multiple people book a self-driving car in small organizations dynamically. In smaller organizations like universities, authorities restrict the movement of vehicles to reduce pollution. So, in order to achieve this, they use electrical carts to carry people and higher authorities. Instead of electric carts, we can introduce self-driving cars for the movement of people in small organizations. So, for this everything should be automated. One of the tasks of automation is a shortest path detection along with accepting different bookings so that the autonomous car can carry the people who are on the way instead of blindly going from one location to another location. In order to achieve this we synthesized an algorithm called Kona's algorithm. Kona's algorithm helps us to prioritize booking in self-driving cars that are used in organizations.

***Key Words*:** Path planning, Search Algorithm, Kona's Algorithm

1. INTRODUCTION

Classical AI plays a vital role for developing systems that rely on finding the path and searching for the shortest path. Even though classical AI is the oldest form of AI, many problems that are related to automation can be solved with classical AI. It contains many algorithms for searching and sorting. For this problem we have many algorithms to find the shortest path in optimal time. Finding the shortest path plays a vital role. But, we need to add features dynamically and it should be done in no time. So that system does not get disturbed for waiting to decide. For this we need to find a search algorithm which has the facility to add features dynamically. We have many techniques and search algorithms that are helpful but we need to do few modifications for those algorithms to meet our requirements. So selecting a basic algorithm which gives optimized results and fast search techniques is the key step.

This study aims to build an algorithm required for finding the shortest path considering the priorities of the user's . Self driving cars which are used in organizations for travelling by considering the priorities of users who intend to travel. The overview of the project is given in the form of flow chart in Fig1 which tells how our system acts. This is very useful for developing systems that act like humans which decide where to go and whom to pick up based on the person's location and priority. So, AI plays a vital role in this. In the following chapters, the related work is studied in detail. In chapter 3, the proposed system and implementation of algorithm and methodology are discussed. In chapter 4, results and discussion are done. In chapter 5, work is concluded and enhancements are discussed.

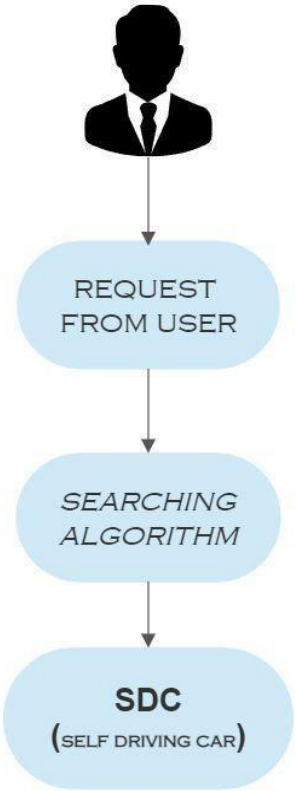


Fig 1: Block Diagram for detecting shortest path

2.RELATED WORK

There are many current works studied by the researchers about path planning for autonomous vehicles. Researchers are mainly focusing on the development of systems for general path planning. Most of the research includes things like motion planning, path planning for shortest paths, perception systems, feedback systems.

Some of the research work helps us to find different algorithms to find shortest paths. Such works of researchers are discussed below,

Obstacle detection in autonomous drive systems is studied in[1].They used the Dijkstra algorithm along with some features of DWA dynamic window which helps for both local path planning and global path planning. The combined results makes the algorithm more feasible and its proved along with results when compared to other algorithms. So, for finding shortest paths this plays a vital role. As it is implemented with hardware, it helps us to prove it is the best for path planning with smooth drive systems.A new Algorithm is developed for the path planning in Autonomous drive systems.The strategies for path planning for indoor environments which are already known are discussed [2].. It gives more accurate output. He analyzed the algorithm and compared it with different path planning algorithms and showed the effectiveness and accuracy of the new algorithm. It is completely based on graphs and analyses the graphs to find shortest paths by considering bugs in the program. So he makes all the analysis of graphs by considering all tracks as bugs. So this is one of the systems which gives the best outcome for path planning where all the results were examined and analyzed in tools of python and graphical analysis is done in this ABUG algorithm.

Many papers were studied on path planning and we have taken only 2 of them which helped us to find which algorithm will be best for finding the path cost.So, only 2 works were taken into consideration.

3.PROPOSED WORK

In this project, We have considered many factors for selecting shortest paths and the algorithm to decide whether to accept the user's request or not. We have designed an algorithm in such a way that there will be no or less effect on the person who was already accepted. This algorithm contains multiple layers to decide whether to accept or to keep the request in the queue. There are 3 steps in this algorithm

3.1 Kona's Algorithm

I. let Ri, : User Request

I1: Initial location of user,

F1 :Final destination of user,

P2 : User priority.

II. If Lf1 <= Lf(I) < Lf(f) :

set Req_Accept =True

Else:

set Req_Accept = False

If L1<=L2

LF = L

If L1>L2

LF = d-L

III. PC(a)=PC1+PC1*(X/Y)

PC(b)=PC2

X=h-P1

Y=h-P

If PC(a)>=PC(b) :

set Req_Accept =True

Else:

set Req_Accept =False

3.2 Terminologies Used

Let L(I): Location factor of initial location of user.

L(f): Location factor of final destination of user

P1: Priority of requested user

P2:Priority of user in bot.

L1 : Level of node where bot is present

L2 : Level of node where final destination of bot

L:level of node

Lf(I): Location factor of initial location of user.

Lf(f): Location factor of final destination of user

Lf1: Location factor of bot location.

PC1=Path cost of shortest path between bot location and final destination of bot

PC2= Path cost of shortest path when user interference is present

h=highest priority of organization.

P2=priority of the user who is in the bot.

3.3 Receiving requests from user

In this process of accepting we take the request from the user we need to take care of our system which sends the request when booked by the user such that it should send the data to our algorithm which contains initial location of the user, final destination of the user, priority of the user (which is defined by the organization). These are the requirements of the algorithm for analysing and detecting whether the path for the request is feasible or not.

3.4 Checking location of the request

This is the initial step where the user request is initially analysed and checks whether the user's path is along the path of the bot. For this requirement we take trees and take a property of trees called levels.

3.4.1 Using levels of tree

Using trees is one of the methods for detecting whether the requested users location and destination are along the path of bot or not using the property of tree called levels. Levels make trees know at what depth they are in. We can use this feature of the trees. We create a tree where it contains all the locations of the organisation in the form of nodes. These nodes have data about locations. These nodes are connected to form a tree. As the locations are connected from one point to another point in the same way nodes are connected. We use trees like general trees because we can connect multiple nodes to it. So the basic tree is now created. Now we assign a factor called 'location factor'. This factor helps us to reverse the levels of nodes in the tree without affecting the tree. This factor can be calculated as

If L1<=L2

Location factor = level of node

If L1>L2

Location factor = depth of tree-level of node

Now we check for the user's initial location and final destination.

If Lf1 <= Lf(I) < Lf(f) :

Then user requests are considered.

Else:

User requests are not considered and added to the queue.

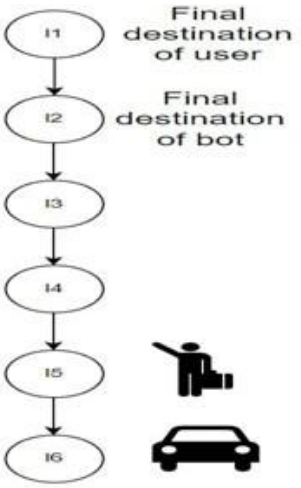


Fig 2:

In fig-2 we have taken an example of a simple tree for testing the step2 of algorithm i.e the level of trees. By observation of the image. L1=6,L2=1. So as L1>L2 then location factor(Lf)=depth of the tree-level of the node.Lets calculate location factors of needed nodes to check whether the user's path is in the direction of the bot path. Lf(l)=1,Lf(f)=5,Lf1=0

As Lf1<Lf(l)<Lf(f). So the path of the user is along the path of the bot. So in this case it's selected and passed to the third step of the algorithm.

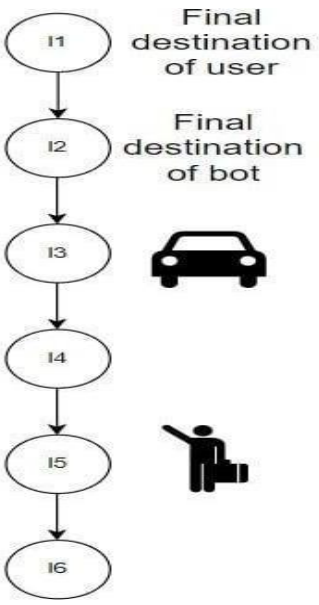


Fig 3:

In fig-3 we have taken an example of a simple tree for testing the step2 of the algorithm is the level of trees. By observation of the image. L1=3,L2=2. So as L1>L2 then location factor(Lf)=depth of the tree-level of the node. Lets calculate location factors of needed nodes to check whether the user's path is in the direction of the bot path. Lf(l)=1,Lf(f)=5,Lf1=3. As Lf(l)<Lf1<Lf(f). So the path of the user is not along the path of the bot. So in this case the user is not selected and waits in the queue.

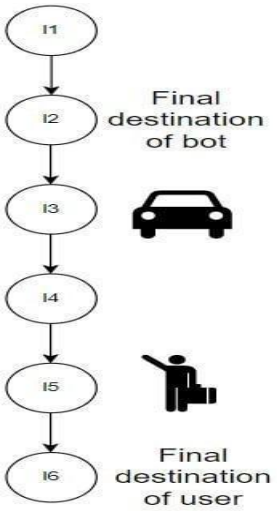
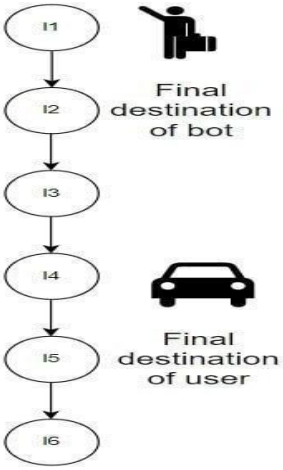


Fig 4:

In fig4 we have taken an example of a simple tree for testing the step2 of algorithm is level of trees. By observation of the image. L1=3,L2=2. So as L1>L2 then location factor(Lf)=depth of the tree-level of the node. Lets calculate location factors of needed nodes to check whether the user's path is in the direction of the bot path. Lf(l)=1,Lf(f)=0,Lf1=3. As Lf(f)<Lf(l)<Lf1. So the path of the user is not along the path of the bot. So in this case the user is not selected and waits in the queue for its turn.



1.

Fig 5:

In fig-5 we have taken an example of a simple tree for testing the step2 of the algorithm is level of trees. By observation of the image. L1=4,L2=2. So as L1>L2 then location factor(Lf)=depth of the tree-level of the node. Lets calculate location factors of needed nodes to check whether the user's path is in the direction of the bot path. Lf(l)=5,Lf(f)=1,Lf1=2.

As Lf(f)<Lf1<Lf(l). So the path of the user is not along the path of the bot. So,in this case the user is not selected and waits in the queue.

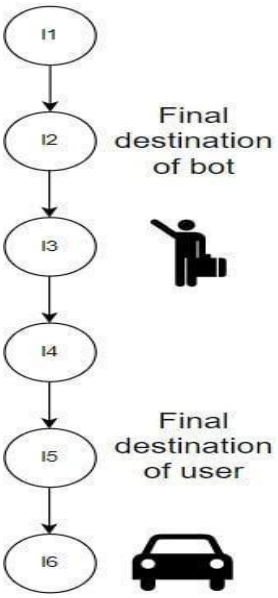


Fig 6:

In fig6 we have taken an example of a simple tree for testing the step2 of the algorithm is the level of trees. By observation of the image. L1=6,L2=2.So as L1>L2 then location factor(Lf)=depth of the tree-level of the node. Lets calculate location factors of needed nodes to check whether the user's path is in the direction of the bot path. Lf(l)=3,Lf(f)=1,Lf1=0

As Lf1<Lf(f)<Lf(l). By this we can determine that path of the user is not along the path of the bot. So,in this case the user is not selected and waits in the queue.

In the above case studies we have seen some examples of situations that will occur and analysed the things and results are shown. From the above analysis Case Study 1 passes the step and moves to step 2 and all other case studies are failed. So,by this analysis we can tell that the step-2 of this algorithm can efficiently find whether the user's path is along the direction of the bot path.

Checking the feasibility of users and bot paths

This is the main part of the algorithm where we check whether the priority of user can balance the extra cost of the path. Whenever a new user is added to the bot, then there will be an increase in path cost due to intervention of user i.e, when ever the bot wants to take the user then it should go pick him up and then need to drop it back so there will be some extra cost for the travel of the bot. Due to this there might be some extra travel to the user who is already present in the bot . So our main agenda is to find whether the priority of the requested user can compensate for this extra path then we consider that this user request is considered.

Here we create nodes with values as location details and we construct a graph for it. Now we use a search algorithm for finding path cost. I want to mention here that there are many search algorithms to find the shortest path. When it comes to searching for shortest paths in maps we generally suggest Dijkstra's algorithm because it is complete where it gives minimal shortest path and it also finds paths whose path cost is less. Time complexity of Dijkstra's algorithm is very less. so all these features make our algorithm use Dijkstra's algorithm. So now lets see how to use the priorities of the user to select whether the user can be picked up or not.

By experimenting with a lot of data with various types of situations we formulated the results in the form of a formula. That is given by

PC(a)=PC1+PC1*(X/Y)
PC(b)=PC2
X=h-P1
Y=h-P2

4. RESULTS AND DISCUSSIONS

In Python 3.7.6, the proposed work is implemented.The dataset for this algorithm was randomly generated.We considered a randomly generated data set and applied an algorithm to it and found the results. Some of the results are shown below.

S.NO	P1	P2	H	PC(A)	PC(B)	RESULT
1	3	3	6	16.27	8.36	YES
2	4	3	10	8.46	7.25	YES
3	4	6	3	3.14	7.35	NO
4	6	2	9	4.78	6.62	NO
5	8	7	5	9.04	6.13	YES
6	6	3	5	0.7976	8.8493	NO
7	6	5	7	2.9976	8.5925	NO
8	7	8	3	6.1347	5.733	YES
9	7	2	8	1.8368	6.1294	NO
10	8	7	5	15.517	8.846	YES

Table 2 : Experimental Results

We noticed that PC1(path cost of shortest path between initial bot location and final location of user in the bot) will always be less than or equal to PC2(path cost of shortest path when user interference is present)

As Kona's algorithm is an algorithm which is used to decide whether to accept the request of the user to be accepted or not without human intervention based on the priority of the user in the organization. Kona's algorithm is categorized into 3 steps. Taking requests from the user, Checking whether the user is in the direction of bot or not using trees. Based on priority of the user given in the organization, the Kona's Algorithm checks whether to select the user or not using graphs by checking whether the priority of the user can affect the increased path cost due to his intervention. Path Cost is calculated with the help of the Dikjistra's Algorithm. If a user is selected then model will send the bot to pick him up else he will added into the queue. This is a brief overview of Kona's Algorithm.

Scenarios

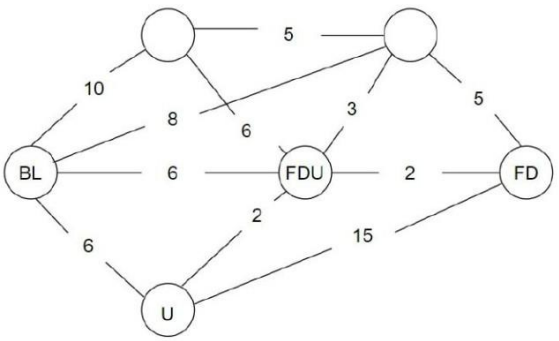


Fig 7:
In Fig 7 we have taken a graph to find whether the user priority can compensate for the extra path cost due to interference of user booking or not.So for it let's follow step 3 of algorithm. Let PC1=8, PC(bi)=6,PC(if)=2,PC(ff)=2. So, PC2=10,h(highest_priority_value_of_organisation)=10,P1=3,

P2=6. By substituting the values in the equation we get PC(a)=22, PC(b)=10 as PC(a)>Pc(b). So users priority is dominating the path cost that is affected by him.

So user request is accepted and the user is taken along the path of bot by rerouting it.

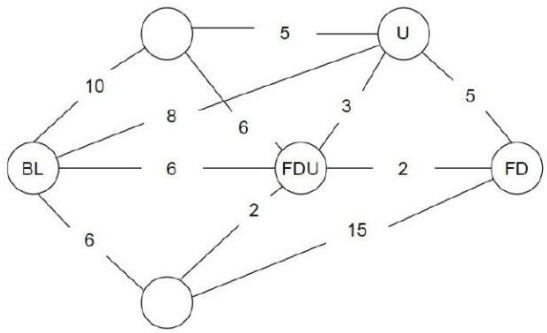


Fig 8:

In Fig 8 we have taken a graph to find whether the user priority can compensate for the extra path cost due to interference of user booking or not. So for it let's follow step-3 of the algorithm. Let PC1=8, PC(bi)=8,PC(if)=2,PC(ff)=2. So, PC2=12. h(highest priority value of organisation) =10 , P1=6,P2=3. By substituting the values in the equation we get PC(a)=9.42, PC(b)=10 as PC(a)<Pc(b). So users priority is unable to dominate the path cost that is affected by him.So user request is not accepted and user is added to the queue.

In above test cases we have seen 2 test cases which tells how priorities of requested users affects the increased path costs and finds whether the users priority can compensate for the path cost increased due to his interference. So we analysed 2 test cases and results were exposed here. In Fig 7 we have been discussing one test case which tells that the user is selected i.e the requested users priority is able to compensate the extra path cost due to users interference.so This user is selected. In fig-8 we took another test case in which users priority can't dominate extra path cost and he is added to the queue and the present transaction will be continued without any disturbance.

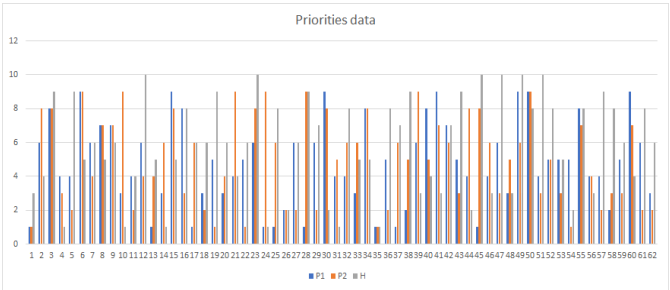


Fig 9:

Figure 9 is a graph showing priorities data taken in the data set.

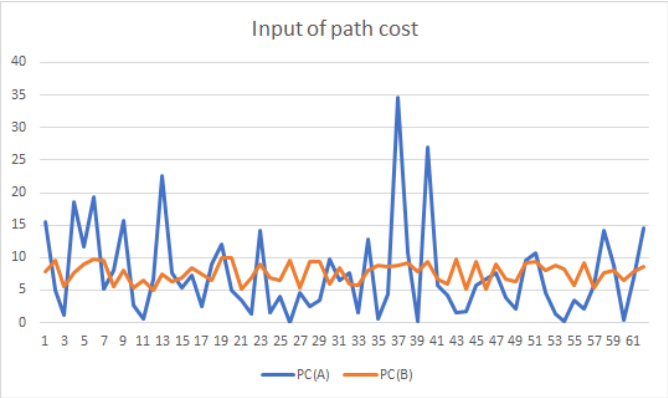


Fig 10:

Figure 10 is a graph showing data of path cost that is calculated by the algorithm in the data set.

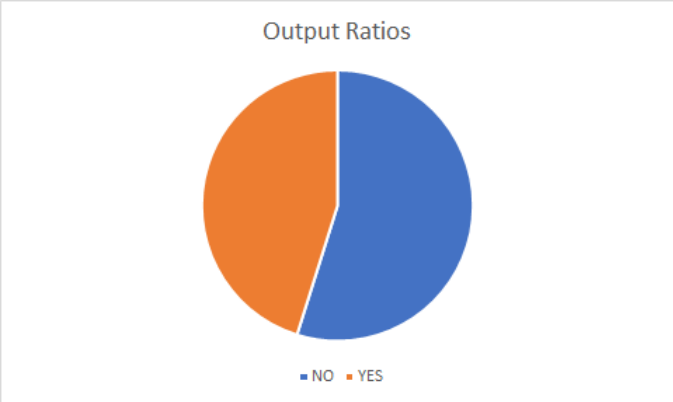


Fig 11:

Figure 11 is having data of output where YES Resembles request accepted and NO resembles reques rejected.

The future scope of this research paper is to build an automatic booking confirmation system in a self-driving car which is not limited to an organization.

5. CONCLUSIONS

Self Driving cars have an autonomous drive system which works without any human intervention. Our algorithm adds a bit of human nature to a self driving car which is choosing to pick a user or not, considering the parameters like location of requested user and final destination of users including the current location of the bot. This nature helps us to use self driving cars in small organizations to travel in organization. We used both tree and graph data structures in this algorithm. We created a synthesized data set in order to check the results. This helps our self-driving system to think like humans whenever it is used in an organization for picking people who

are in the organization based on the priority of the person in the organization.

REFERENCES

1) Li-Sang Lui, jia-feng Lin, ”Path planning for Smart Car Based on Dijkstra Algorithm and Dynamic Window Approach ". Published in Hindawi Journals on 15th feb 2021.
2) Rimon, E., and Koditschek, D. E. 1992. Exact robot navigation using artificial potential fields. IEEE Transactions on Robotics and Automation 8(5):501–518
3) Connolly, C.; Burns, J.; and Weiss, R. 1990. Path planning using laplace’s equation. In IEEE International Conference on Robotics and Automation (ICRA), 2102–2106.
4) Hoang van Dung, Kang-Hyun Jo, “Path planning for autonomous drive Vehicles on heuristic using online images”. Vietnam journal of computer science May 2014
5) “Phantom auto to be operated here,”The Free-Lance Star (newspaper distributed throughout Fredericksburg, Virginia, USA), June. 18 1932.
6) Hao Ma, Wenhui Pei1, and Qi ZhangHao Ma , Wenhui Pei and Qi Zhang “Research on Path Planning Algorithm for Driverless Vehicles”
7) iajia Chen,[1]Rui Zhang,[1]Wei Han,[1]Wuhua Jiang,[1]Jinfang Hu,[1]Xiaoshan Lu,[1]Xingtao Liu,[1]and Pan Zhao ,”Path Planning for Autonomous Vehicle Based on a Two-Layered Planning Model in Complex Environment”,published in Hindawi Journal,27th nov,2020
8) C. Urmson, J. Anhalt, D. Bagnell et al., “Autonomous driving in urban environments: boss and the urban challenge,” *Journal of Field Robotics*, vol. 25, no. 8, pp. 425–466, 2008
9) N. Ganganath and C. T. Cheng, “A 2-dimensional ACO-based path planner for off-line robot path planning,” in *Proceedings of the International Conference on Cyber-Enabled Distributed Computing & Knowledge Discovery*, IEEE, Beijing, China, October 2013.
10) A. Stentz, “Optimal and efficient path planning for partially-known environments,” in *Proceedings of the IEEE International Conference on Robotics & Automation*, IEEE, Washington, DC, USA, May 2002.
11) DmitriDolgov,Sebastianhrun,MichaelMontemerlo,James Diebel Practical Search Techniques in Path Planning for Autonomous Driving, Aritcle on path planning by Stanford.
12) Choset, H., and Burdick, J. 2000. Sensor-based exploration: The hierarchical generalized voronoi graph. The International Journal of Robotics Research 19.
13) Koren, Y., and Borenstein, J. 1991. Potential field methods and their inherent limitations for mobile robot navigation. In ICRA. LaValle, S. 1998. Rapidly-exploring random trees: A new tool for path planning.
14) Tilove, R. 1990. Robotics and automation. In IEEE International Conference on Robotics and Automation, volume 1, 566 – 571
15) Kavraki, L.; Svestka, P.; Latombe, J.-C.; and Overmars, M. 1996. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE Transactions on Robotics and Automation.
16) “Path Planning for Autonomous Vehicles with Hyperloop Option”,blog by intellipas